



國立清華大學
NATIONAL TSING HUA UNIVERSITY

Deep Learning and Applications

Ching-I Huang

Learning

- Aim to learn a function that maps inputs x to target y by the weight a and bias b :

$$y = ax + b$$

- Also write it as:

$$y = b + wx$$

- The goal of learning is to find parameters w and b that minimize the **loss function** L , which measures the error between predicted values $\hat{y}$ and actual outputs y .

$$L = \frac{1}{N} \sum_n e_n \quad e_n = (y_n - \hat{y}_n)^2$$

- Optimization Goal

Refine the function such that its estimates progressively approach the true values.

$$(w^*, b^*) = \arg \min_{w, b} L(w, b)$$

Polynomial Curve Fitting

- Our goal is to exploit this training set in order to make predictions of the value $\hat{t}$ of the target variable for some new value of $\hat{x}$ input variable.
- Polynomial Curve Fitting

$$y(x, \mathbf{w}) = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M = \sum_{j=0}^M w_jx^j \quad (1.1)$$

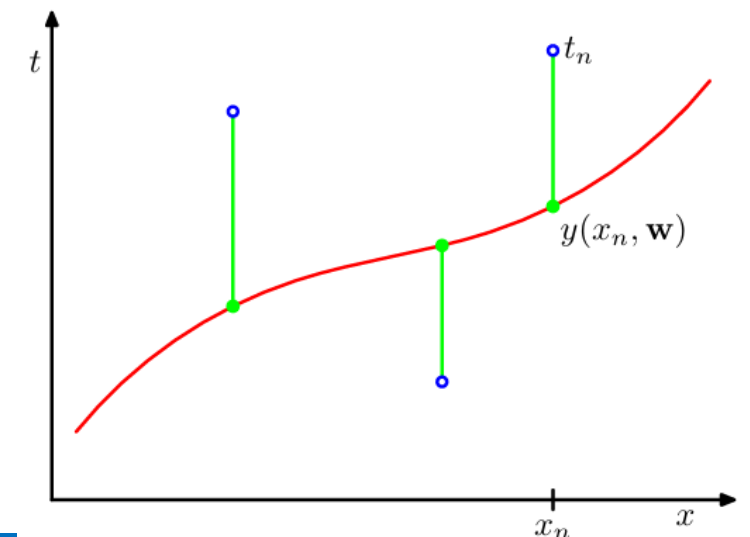
The values of the coefficients will be determined by fitting the polynomial to the training data.

- Error function: the misfit between the function $y(x, \mathbf{w})$, for any given value of $\mathbf{w}$, and the training set data points.

ex: the sum of the squares of the errors between the predictions

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{ \overset{\text{predictions}}{y(x_n, \mathbf{w})} - \overset{\text{target}}{t_n} \}^2 \quad (1.2)$$

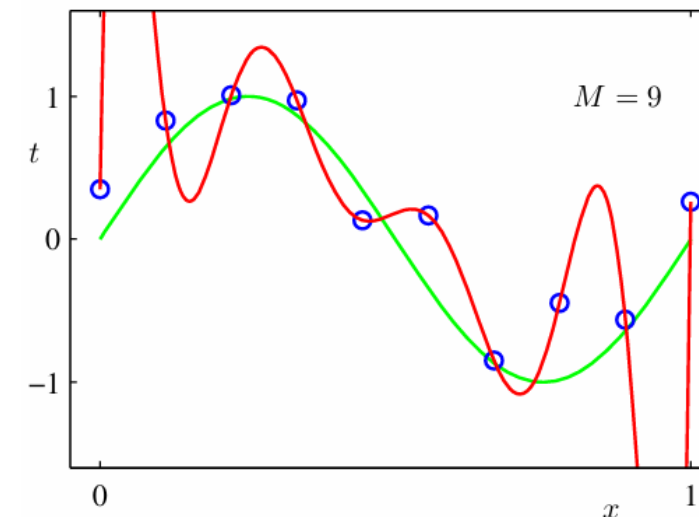
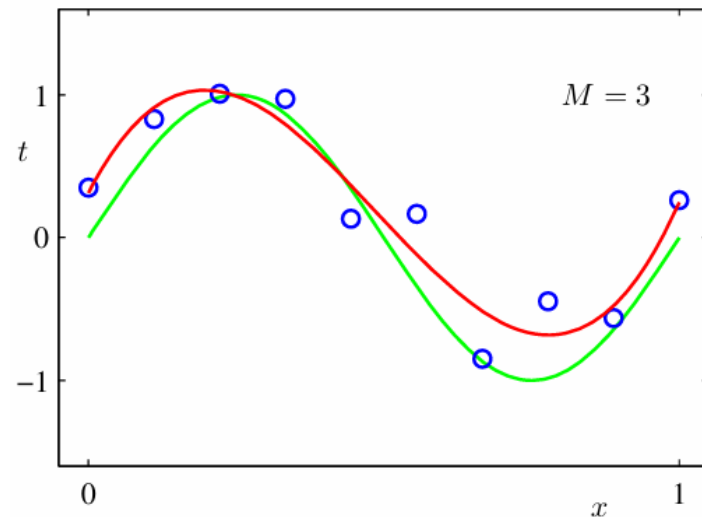
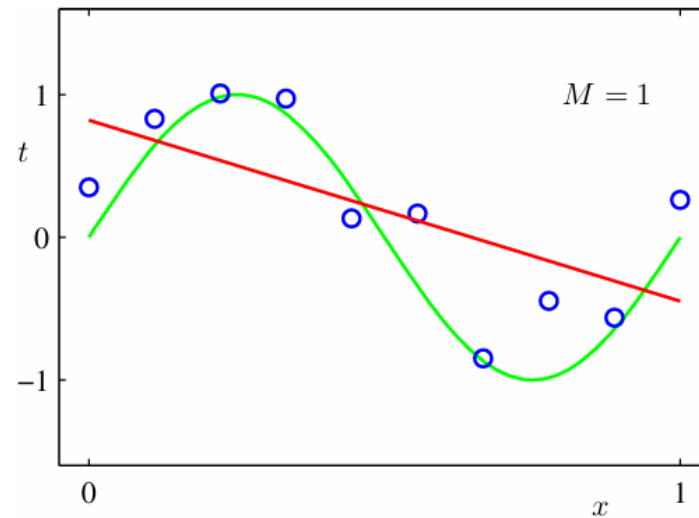
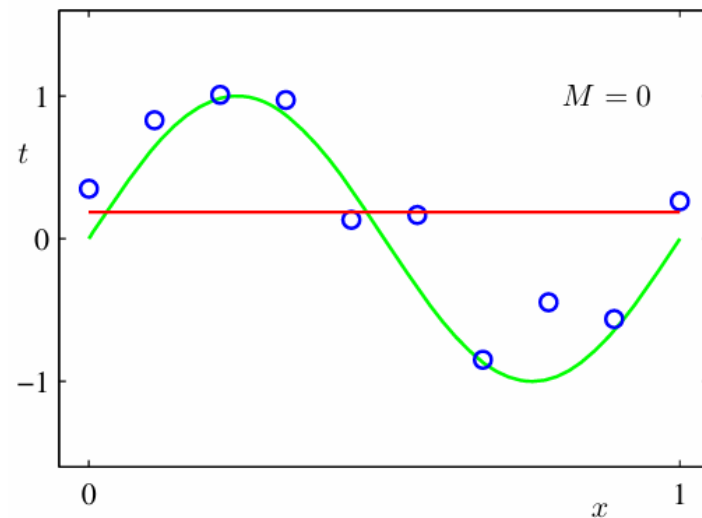
- The resulting polynomial is given by the function $y(x, w^*)$.





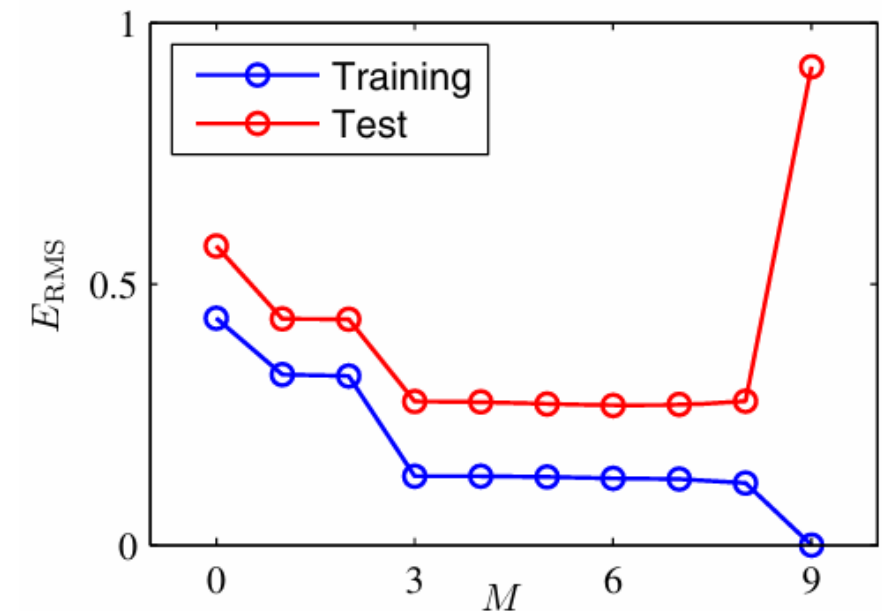
Polynomial Curve Fitting

- Model selection



The polynomial when $M = 9$ passes exactly through each data point and $E(\mathbf{w}^*) = 0$. However, the fitted curve oscillates wildly and gives a very poor representation of the function

over-fitting: the fitted curve oscillates wildly and gives a very poor representation of the function $\sin(2\pi x)$





Benefits form Large-Scale Dateset

- For a given model complexity, the over-fitting problem become less severe as the size of the data set increases.

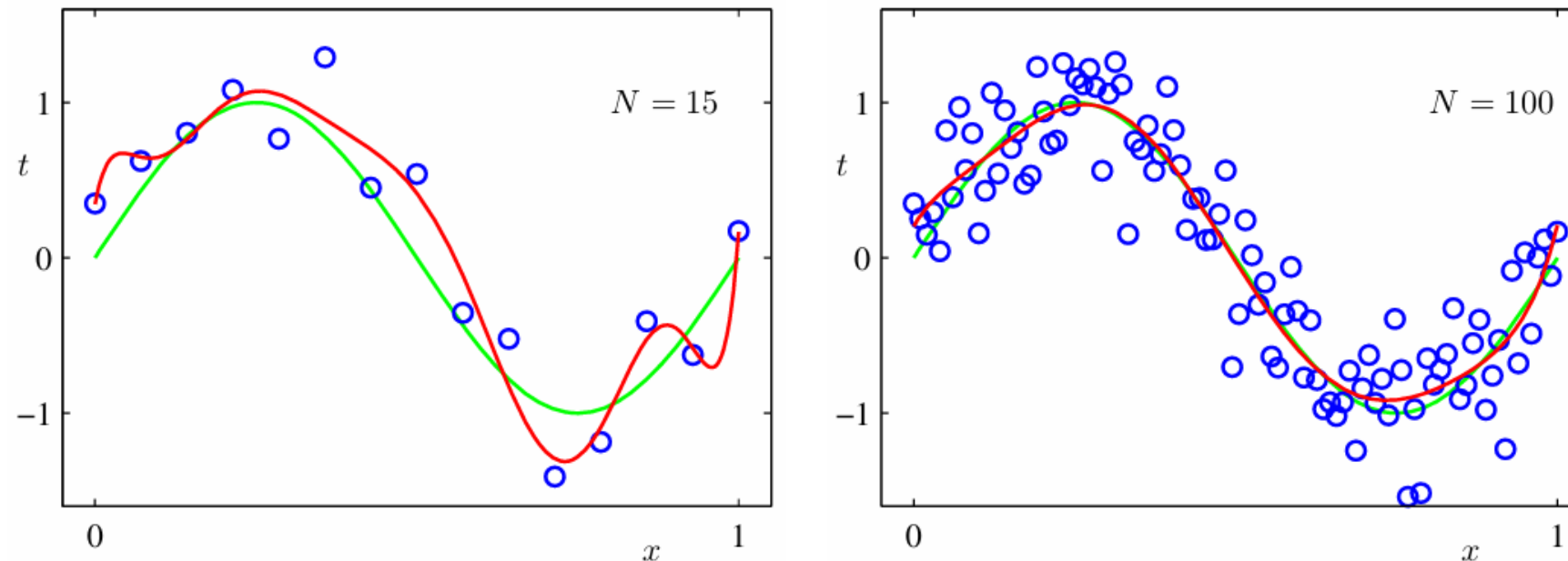


Figure 1.6 Plots of the solutions obtained by minimizing the sum-of-squares error function using the $M = 9$ polynomial for $N = 15$ data points (left plot) and $N = 100$ data points (right plot). We see that increasing the size of the data set reduces the over-fitting problem.

Having to limit the number of parameters in a model according to the size of the available training set

→ Choose the complexity of the model according to the complexity of the problem being solved

Find the model parameters represents a specific case of maximum likelihood

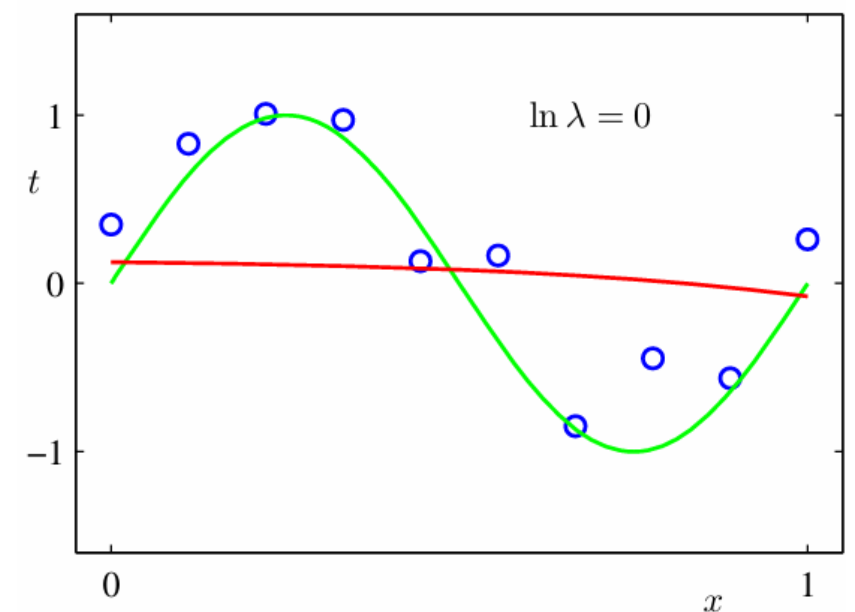
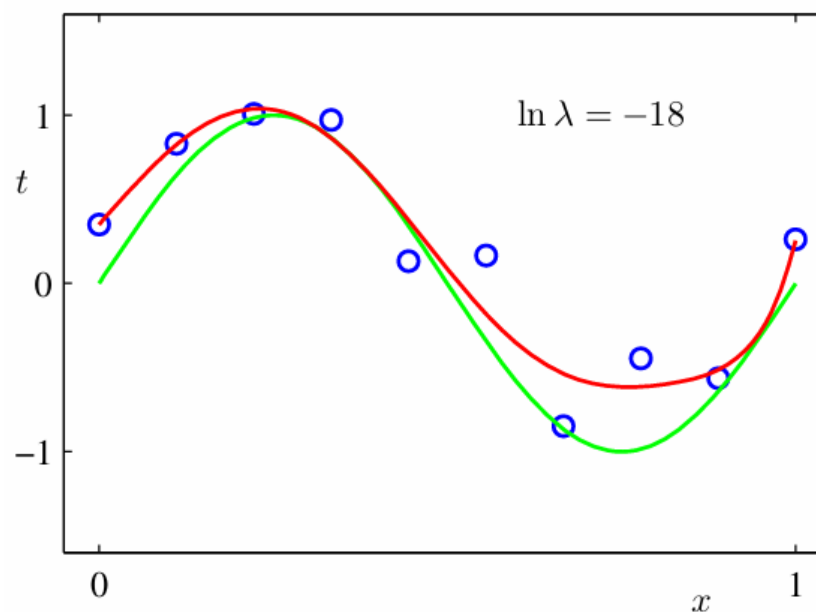
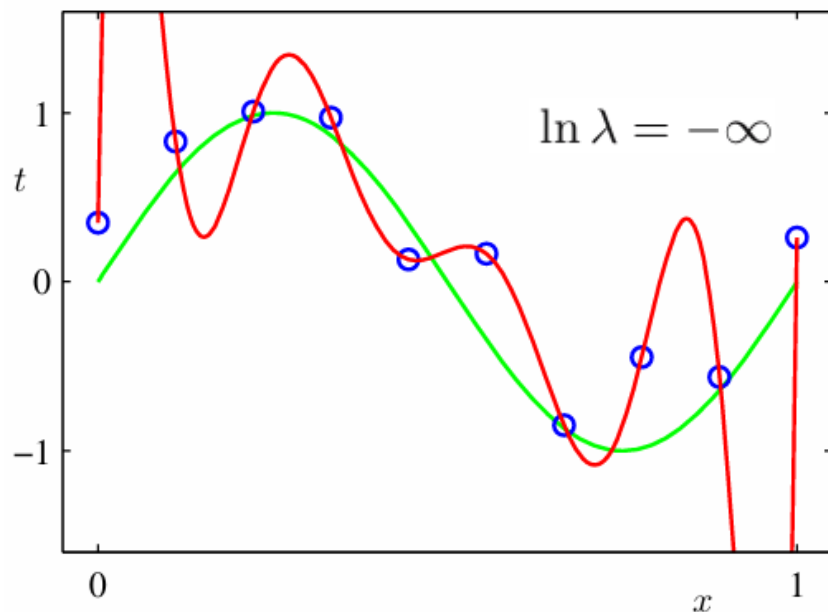


Discourage the Coefficients from Reaching Large Values

- A modified error function

$$\tilde{E}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2 + \underbrace{\frac{\lambda}{2} \|\mathbf{w}\|^2}_{\text{penalty term / Importance/ weighted coefficient}} \quad (1.4)$$

where $\|\mathbf{w}\|^2 \equiv \mathbf{w}^T \mathbf{w} = w_0^2 + w_1^2 + \dots + w_M^2$



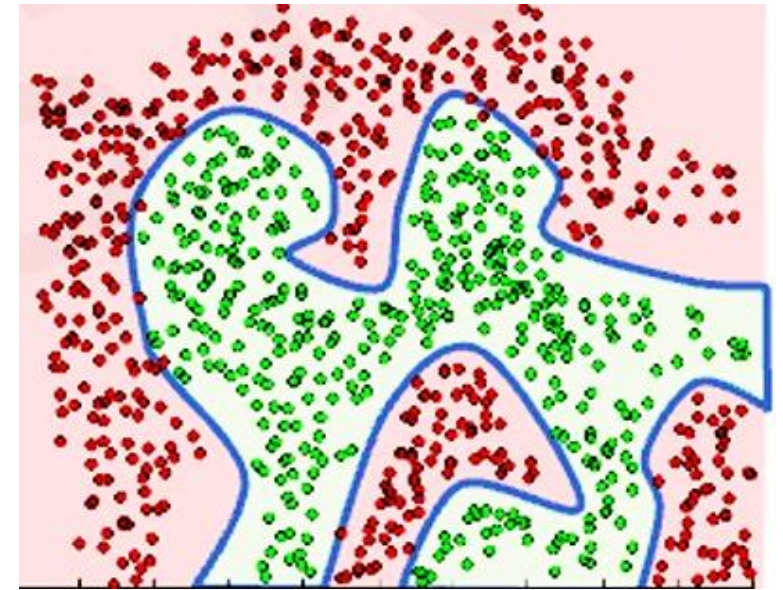
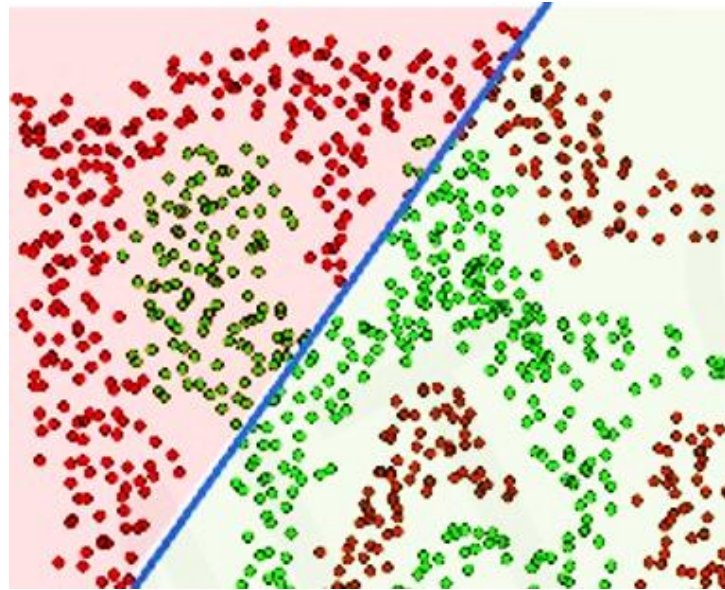
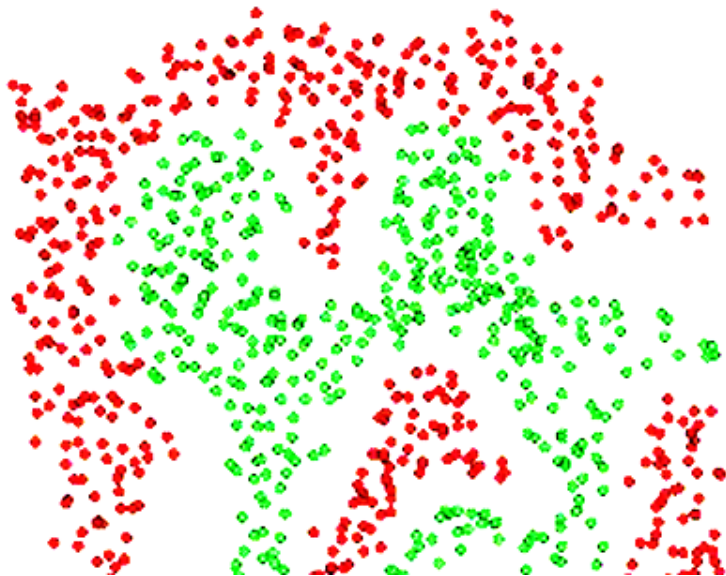


Linear Model

- A linear combination of the input variables

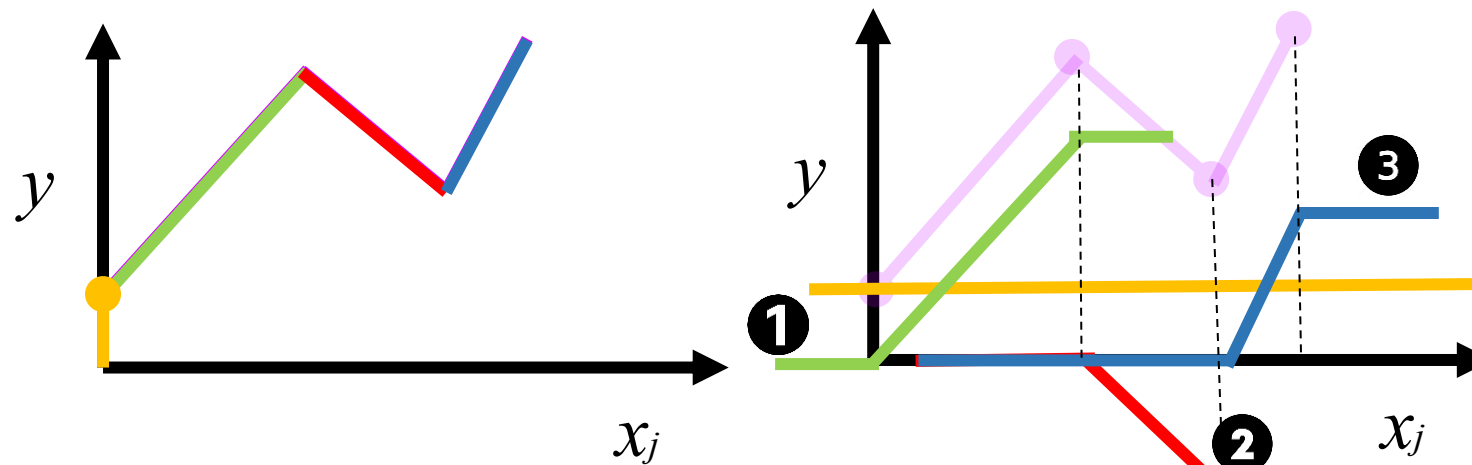
$$y(\mathbf{x}, \mathbf{w}) = w_0 + w_1x_1 + \dots + w_Dx_D \quad \text{where } \mathbf{x} = (x_1, \dots, x_D)^T.$$

- Model bias: the presence of systematic errors in a model that can cause it to consistently make incorrect predictions
→ Add some features for nonlinearity



Activation Functions

- To the output of a linear transformation yields a nonlinear transformation, activation function g is typically chosen to be a function that is applied element-wise.



$$y = \dots + \text{green line} + \text{red line} + \text{blue line}$$

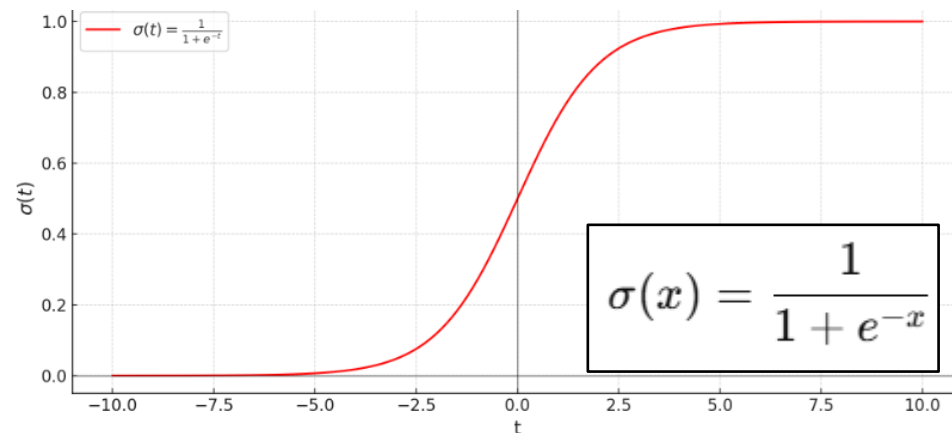
$$= b + \sum \text{black line}$$

$$y = b + \sum_j w_j x_j \quad j: \text{number of features}$$

$$= b + \sum_i c_i g(b_i + \sum_j w_{ij} x_j)$$

i : number of sigmoid functions

- Sigmoid function



$$\sigma(b + wx_j) = c \cdot \frac{1}{1 + e^{-(b+wx_j)}}$$

c related to height
 b related to shift
 w related to slope

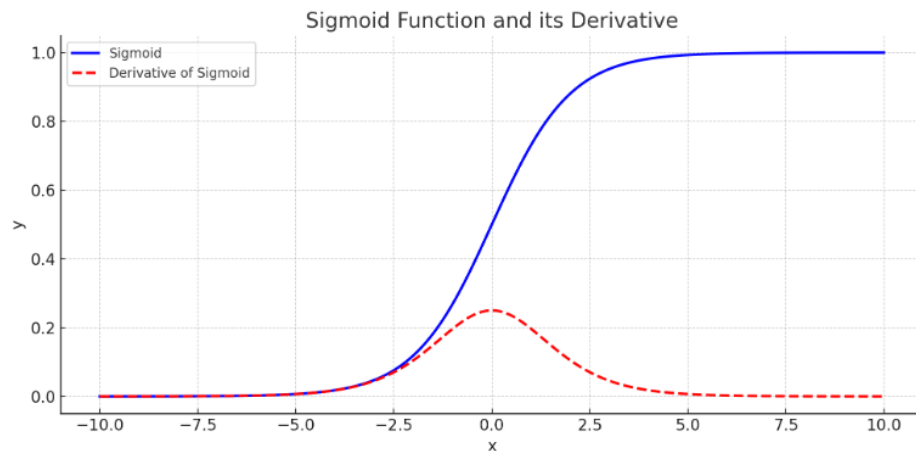


Activation Functions

- Common Activation Functions

Sigmoid Function

 `tf.math.sigmoid(z)`

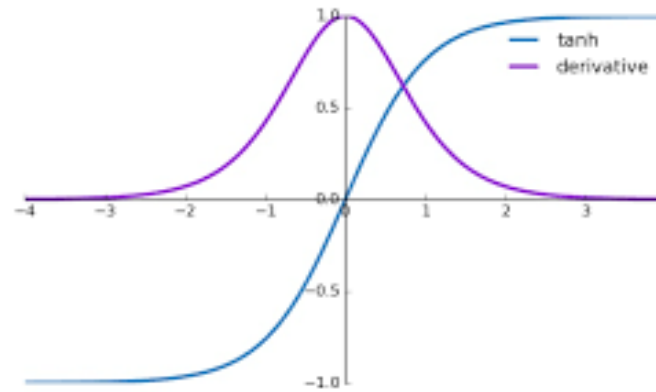


$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z))$$

Hyperbolic Tangent

 `tf.math.tanh(z)`

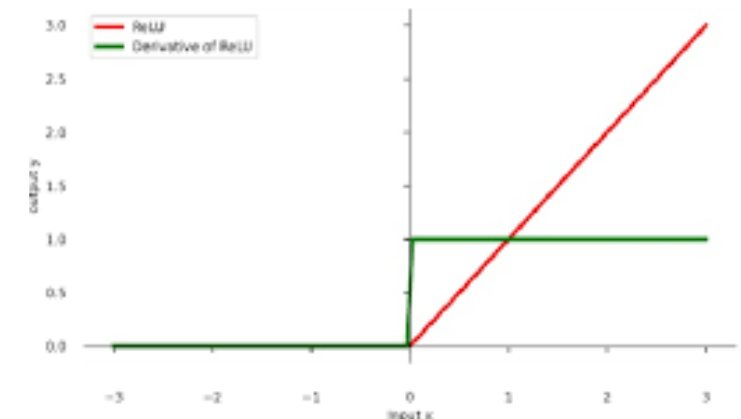


$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$g'(z) = 1 - g(z)^2$$

ReLu (Rectified Linear Unit)

 `tf.nn.relu(z)`



$$g(z) = \max(0, z)$$

$$g'(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$$